3

Handling Missing Data

April, 2026

Rachid BENOUINI

Missing data is everywhere

Survey data

- In favor: 30%, Opposed: 50%, The rest: Non-response!

Research studies and clinical trials

- Subject drop out or censoring

Recommendation systems

- Different individuals interact with or express preferences for different items

Data integration

- Different variables collected (with different frequencies) by different organizations or in different experiments or trials

Definitions

- Missing data
  - Meaning the absence of values for certain observations, which is an unavoidable problem in most data sources.

- Can machine learning model handle missing data ?
  - Some model implementations can it out of the box.
  - For other, we must do some data cleaning.

- Imputation:
  - The act of replacing missing data with their statistical estimates.
  - The goal of this technique is to produce a ***complete dataset***.

Types of Missing Data (Missing Mechanisms)

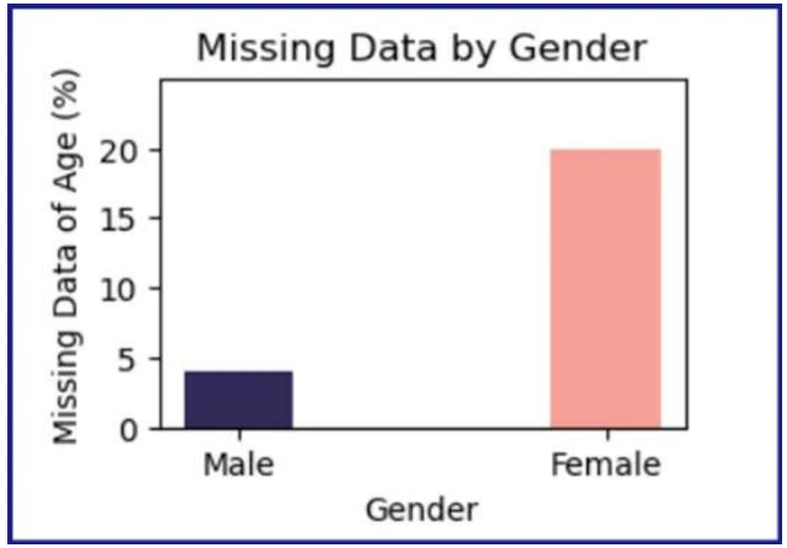
- MCAR (Missing Completely at Random)
  - Data is missing in a completely random manner, meaning the probability of a value being missing is independent of both observed and unobserved data. There is no systematic pattern causing the missing values.
- Example:
  - If some participants in a survey randomly skip questions due to distraction or confusion
  - A sensor randomly malfunctions and fails to record temperature readings at certain times.

Product Survey Data

Age	Buy Again?	Recommend	Satisfaction
19	4	3	4
26	5		5
14	4	3	3
39	5	4	
45	3	4	4
52	4	3	4
36	2	1	1

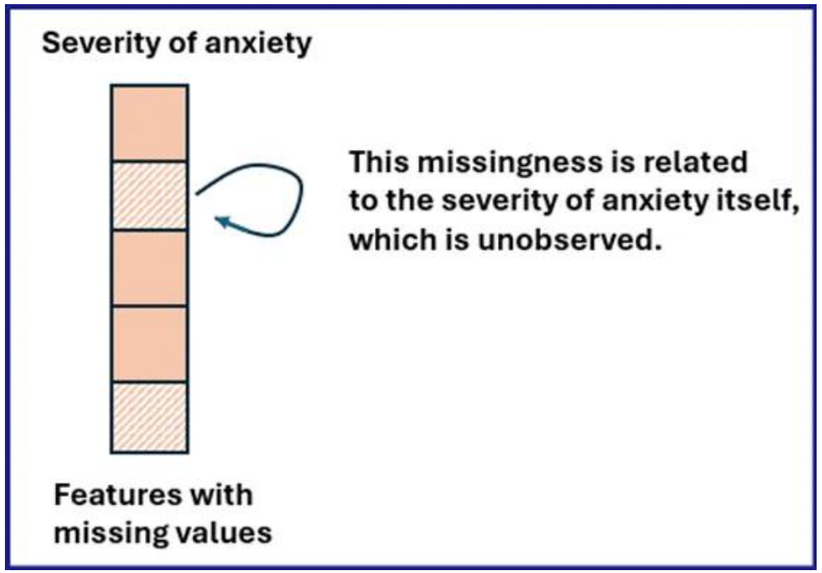
Types of Missing Data (Missing Mechanisms)

- MAR (Missing at Random)
- The probability of missing data depends only on observed data but not on the missing values themselves.
- In other words, missingness is systematically related to some known variables but not to the missing variable's true values.
- Example:
  - female participants in a survey are less likely to report their age compared to male participants
  - In a customer survey, younger individuals are less likely to disclose their income, but older individuals tend to answer the question. Here, missingness depends on the age variable (which is observed).
  - Patients with higher cholesterol levels are more likely to have their blood sugar tested, leading to more missing values in the blood sugar column for people with low cholesterol.



Types of Missing Data (Missing Mechanisms)

- MNAR (Missing Not at Random)
  - The probability of missing data depends on the value of the missing data itself, meaning there is a systematic reason why certain values are missing. This is the most problematic type of missingness because it introduces bias.
- Example:
  - In a mental health survey, individuals with severe anxiety may be less likely to answer questions about their mental health, making the missing responses directly influenced by the severity of their condition.

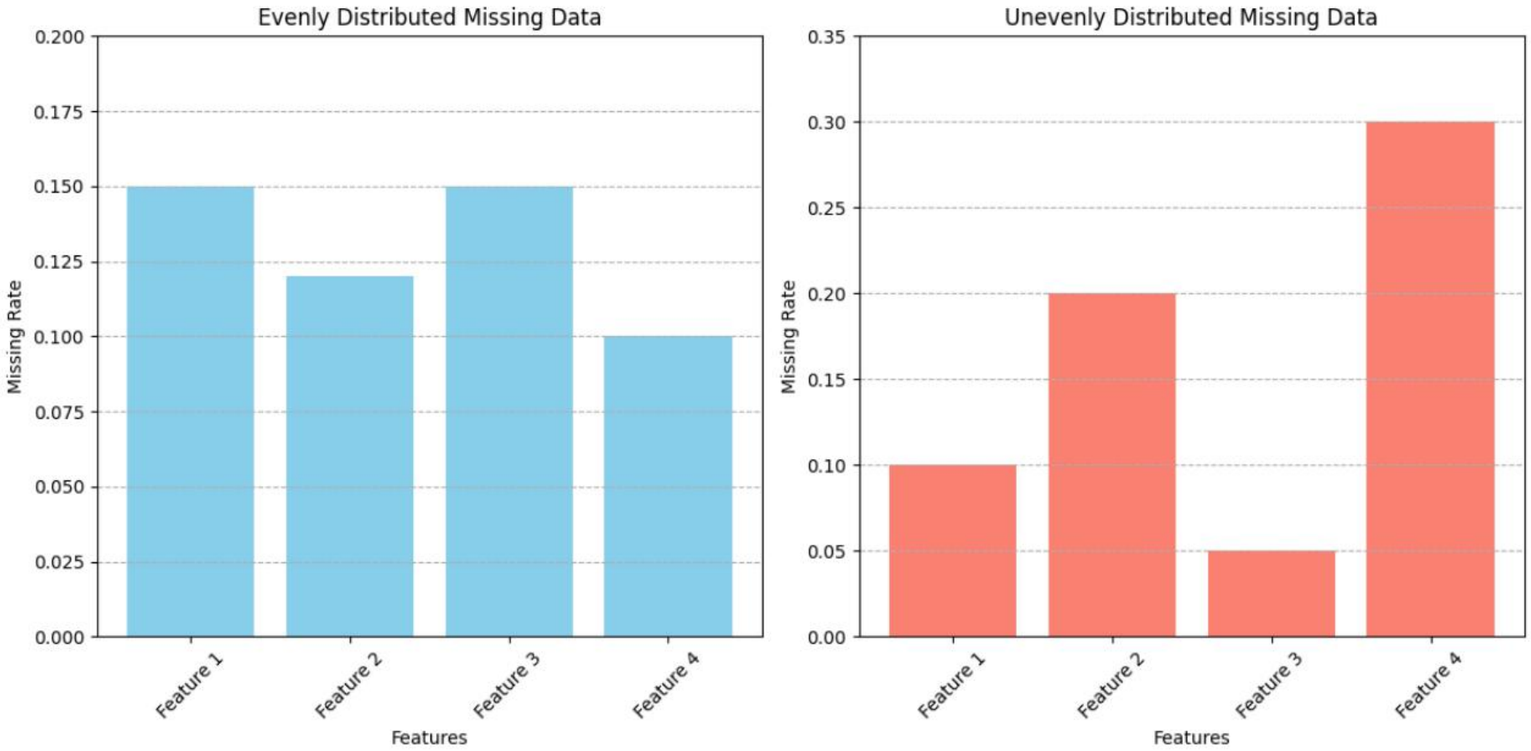


Summary of types of Missing Data

Missing Type	Cause of Missingness	Example	Implication
MCAR (Missing Completely at Random)	Random, no relationship to any data	A sensor randomly fails to record temperature.	No bias; removing missing rows is acceptable if % is low.
MAR (Missing at Random)	Related to observed variables but not the missing values themselves	Younger people are less likely to disclose income.	Can be handled with imputation using observed data.
MNAR (Missing Not at Random)	Related to the missing values themselves	High-income individuals are less likely to report income.	High risk of bias; requires advanced techniques or external data.

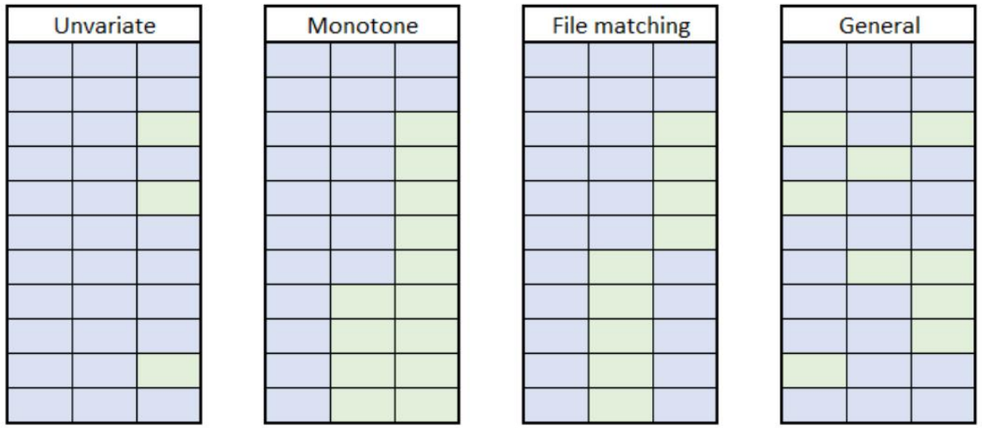
Missing Rate

- The missing rate refers to the proportion of missing data in the entire dataset. It plays a crucial role in determining the difficulty of solving the missing data problem.
- A higher missing rate indicates that a larger portion of the data is missing, resulting in less information available for algorithms to analyze, thereby making the task more challenging.



Missing Pattern

- **Univariate and Multivariate Patterns:**
  - In the Univariate pattern, missing values occur in only one variable or column of the dataset. For instance, in a dataset containing information about age, income, and education, the univariate missing pattern occurs when one of these variables has missing values, while the others have complete data
- **Monotone and Non-Monotone Patterns:**
  - In a monotone missing pattern, missing values occur systematically in one direction within the dataset. This implies that once a value is missing, all subsequent values in the dataset are also missing.
- **Connecting and Unconnecting Patterns:**
  - File matching depicts a connecting pattern, where it is possible to travel to all blue cells by horizontal or vertical moves. However, this pattern becomes unconnected if we remove the first column.



Existing Methods of Handling Missing Data

There are three main approaches to handling missing data:

Representation Learning

- Uses ML models to learn meaningful representations from raw data, improving the imputation process by capturing complex relationships.
- Enhances accuracy and robustness and is sometimes combined with other imputation techniques.

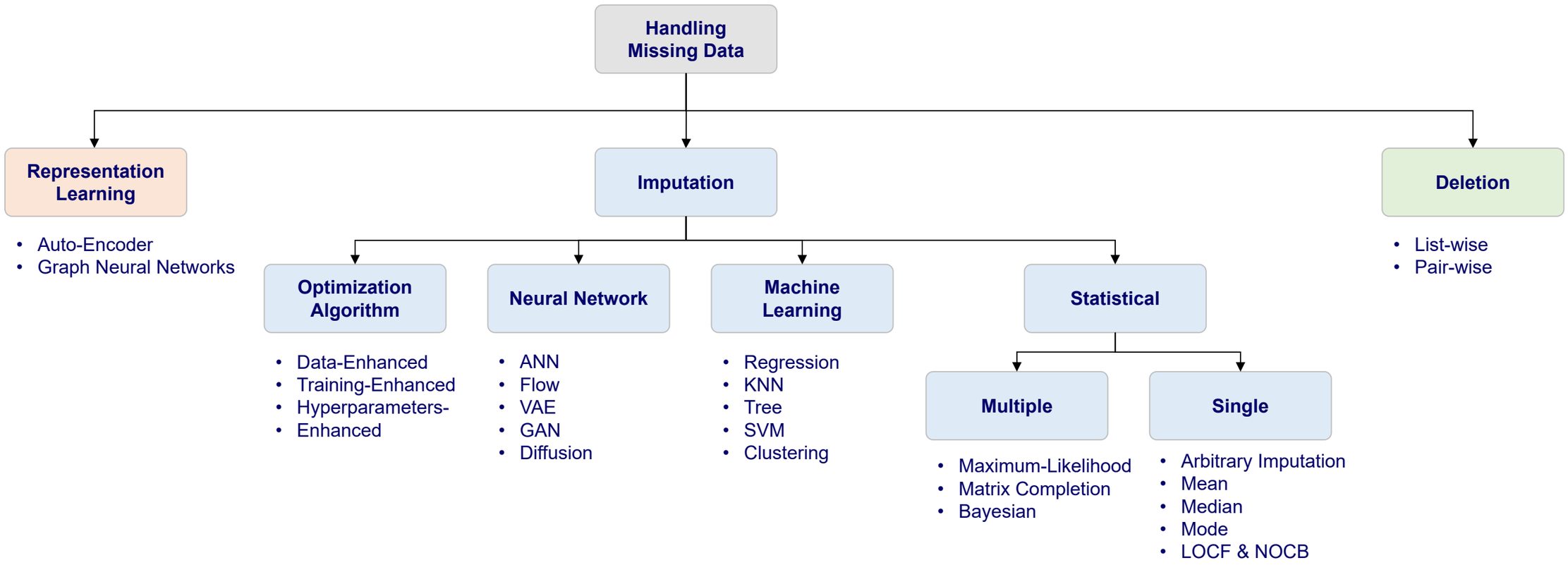
Imputation

- Fills in missing values using techniques such as mean imputation, regression, and machine learning.
- Preserves dataset integrity and reduces biases from data removal.
- A key area of focus is neural network-based imputation, which has gained popularity over time.

Deletion

- Involves removing rows or columns with missing values.
- Very simple.
- Can lead to biased results and loss of valuable information, especially when missing data follows specific patterns.

Taxonomy of Existing Methods of Handling Missing Data



Next generation	Current Trend	Effective	Simple	Easy

Listwise Deletion - Deleting Data Points with Missing Values

This method removes entire rows with any missing values, allowing analyses to be conducted on complete datasets. It is also called Complete Case Analysis (CCA) meaning analysing only those observations in the dataset that contain values in all the variables.

While it simplifies analysis and maintains consistent sample sizes, it can lead to significant data loss and potential bias if the missingness is not completely random. Deleting data points can substantially reduce the dataset size, resulting in inadequate training and poorer model performance.

Datapoints: 7; Features: 4

Age	Buy Again?	Recommend	Satisfaction
19	4	3	4
26	5		5
14	4	3	3
39	5	4	
45	3	4	4
52	4	3	4
36	2	1	1

Drop rows with missing values

Datapoints: 5; Features: 4

Age	Buy Again?	Recommend	Satisfaction
19	4	3	4
14	4	3	3
45	3	4	4
52	4	3	4
36	2	1	1

```
train.dropna(subset=['Embarked'],how='any',inplace=True)
```

Listwise Deletion - Dropping Features with Missing Values

Another strategy for handling missing data is to identify and remove features with a high proportion of missing values, typically over 80%.

However, it is important to retain valuable features that contribute to the outcome. Instead of discarding entire rows, selectively removing features with excessive missingness is often preferable, especially in larger datasets where the missingness may not be informative. This approach helps maintain a robust dataset, enhancing analysis and potentially improving model performance.

Datapoints: 7; Features: 4

Age	Buy Again?	Recommend	Satisfaction
19	4	3	4
26	5		5
14	4	3	3
39	5		
45	3	4	4
52	4		4
36	2		1

Drop features with missing values

Datapoints: 7; Features: 3

Age	Buy Again?	Satisfaction
19	4	4
26	5	5
14	4	3
39	5	
45	3	4
52	4	4
36	2	1

```
train.drop(columns=['Cabin'], axis = 1, inplace = True)
test.drop(columns=['Cabin'], axis = 1, inplace = True)
```

Pairwise Deletion

Handles missing values by removing cases containing missing values only when the affected features are used in analyses.

For example, if the *1*-st instance has a missing value in the *j*-th column, the *1*-st instance will be excluded only when analyzing the jth feature.

However, it will be included in analyses involving other features.

List wise deletion

Gender	Manpower	Sales
M	25	343
~~F~~	~~.~~	~~280~~
M	33	332
~~M~~	~~.~~	~~272~~
~~F~~	~~25~~	~~.~~
M	29	326
~~	~~26~~	~~259~~
M	32	297

Pair wise deletion

Gender	Manpower	Sales
M	25	343
F	~~.~~	280
M	33	332
M	~~.~~	272
F	25	~~.~~
M	29	326
~~	26	259
M	32	297

Lab - Managing missing data with pandas

Data URL:

https://raw.githubusercontent.com/kjam/data-cleaning-101/master/data/iot_example_with_nulls.csv

```
import pandas as pd
df = pd.read_csv(
    "https://raw.githubusercontent.com/kjam/data-cleaning-101/master/data/iot_example_with_nulls.csv",
    na_values=["n/a"]
)
df.head(10)
```

	timestamp	username	temperature	heartrate	build	latest	note
0	2017-01-01T12:00:23	michaelsmith	12.0	67	4e6a7805-8faa-2768-6ef6-eb3198b483ac	0.0	interval
1	2017-01-01T12:01:09	kharrison	6.0	78	7256b7b0-e502-f576-62ec-ed73533c9c84	0.0	wake
2	2017-01-01T12:01:34	smithadam	5.0	89	9226c94b-bb4b-a6c8-8e02-cb42b53e9c90	0.0	NaN
3	2017-01-01T12:02:09	eddierodriguez	28.0	76	NaN	0.0	update

```
# compute the % of missing values
df.isnull().sum()*100 /df.shape[0]
```

	0
timestamp	0.000000
username	0.000000
temperature	22.102229

Lab - Managing missing data with pandas

```
# delete a colum that contains more than 30% of missing data
df_dropped = df.drop(columns=["note"], axis=1)
df_dropped.head(10)
```

```
# Drop rows with any missing values
df_dropped = df.dropna()
df_dropped.head(10)
df_dropped.isnull().sum()*100 /df_dropped.shape[0]
```

```
# Now we replace the missing values in the temperature column with the mean value
temp_mean = df["temperature"].mean()
fill_mean = df["temperature"].fillna(temp_mean)
fill_mean.value_counts()
```

Imputation - Univariate - Arbitrary/Constant Value Imputation

Filling missing values with a fixed predefined value, e.g., 0, "Unknown", or a placeholder

When to use:

- a missing value has a specific meaning.
- you want to **preserve the missingness** pattern for later analysis.
- working with categorical or ordinal variables where **no logical imputation is possible**.

Example:

- Replacing missing values in a job title column with "Unknown" instead of imputing based on existing data.

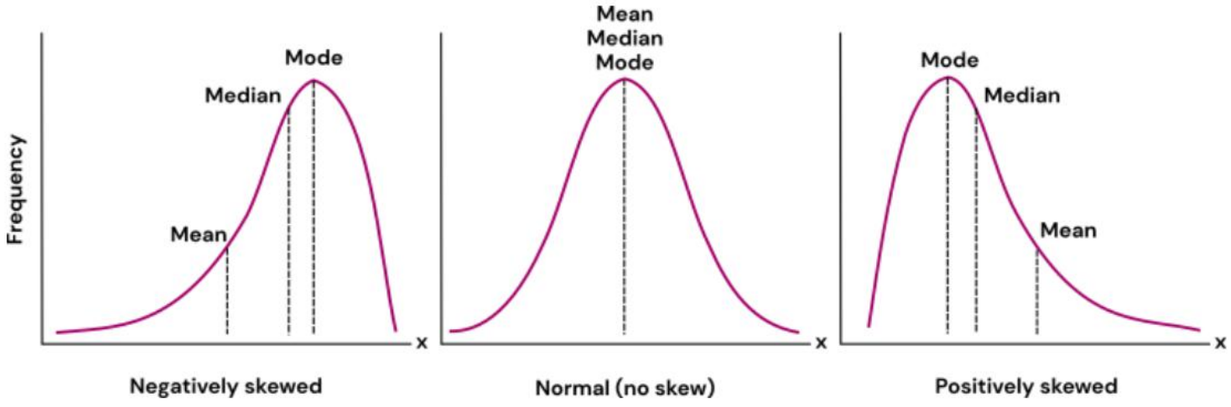
Limitations:

- Can introduce **bias** if the chosen constant is not representative.
- May mislead models into **treating the placeholder as a real category**.

```
# using pandas
train['Embarked'] = train['Embarked'].fillna('S')
```

Imputation - Univariate - Mean, Median and Mode

Method	When to Use	Example	Limitations
Mean	• Data is **normally distributed** (no significant skew). • Missing values are randomly distributed (MCAR or MAR).	• Imputing missing exam scores in a dataset where scores are **normally distributed**.	• **Not suitable for skewed data** (e.g., income, house prices). • Can **reduce variance**, leading to biased estimates.
Median	• Data is **skewed** (e.g., income, property prices). • **Less sensitive** to outliers.	• Imputing missing household incomes, where a few billionaires heavily skew the distribution.	• May **not accurately represent the dataset** if data is normally distributed.
Mode	• Dealing with **categorical data** or **discrete variables** (e.g., gender, occupation, product category).	• Imputing missing "Preferred Payment Method" in a customer dataset where most people choose "Credit Card".	• Can **distort proportions** in highly imbalanced datasets. • **Only useful for categorical** variables.



Median Imputer

```
# calculating median age of Training data
median_age = df['Age'].median()
df['Age'].fillna(median_age, inplace=True)
```

Imputation - Univariate - Random Value Imputation

Filling missing values with randomly sampled values from the existing data distribution

When :

- you want to **maintain natural variability** if data is missing completely at random (MCAR).

Example:

- Imputing missing house prices by randomly sampling from the observed distribution rather than using a fixed mean/median.
- May introduce artificial noise, especially if missing data is not truly random.

Limitations:

- **Not suitable** when missing not at random (MNAR).

Random Imputer

```
# Impute missing values in train
missing_train = train['Age'].isnull().sum()
if missing_train > 0:
    samples_train = train['Age'].dropna().sample(missing_train, random_state=42).values
    train.loc[train['Age_imputed'].isnull(), 'Age_imputed'] = samples_train
```

Imputation - Univariate - LOCF & NOCB (for Time Series Data)

When variables are repeatedly measured over a series of time points or observation have a sequential relationship.

```
Date
2023-10-22    12103.17
2023-10-29         NaN
2023-11-05         NaN
2023-11-12    13987.57
2023-11-19         NaN
2023-11-26    13560.20
2023-12-03    13640.13
2023-12-10    12554.56
2023-12-17    14908.75
2023-12-24    14325.40
2023-12-31    14818.50
Name: Sales, dtype: float64
```

Last observation carried forward (LOCF)

- Fills in missing values with the last known value, so if a value is missing, it uses the most recent one before it.

Next observation carried backward (NOCB)

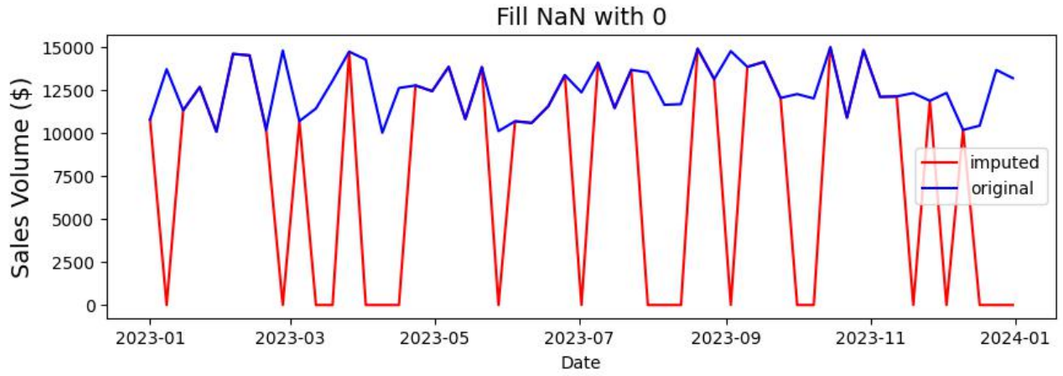
- Fills missing values with the next known value in the column, so if a value is missing, it uses the next one after it.

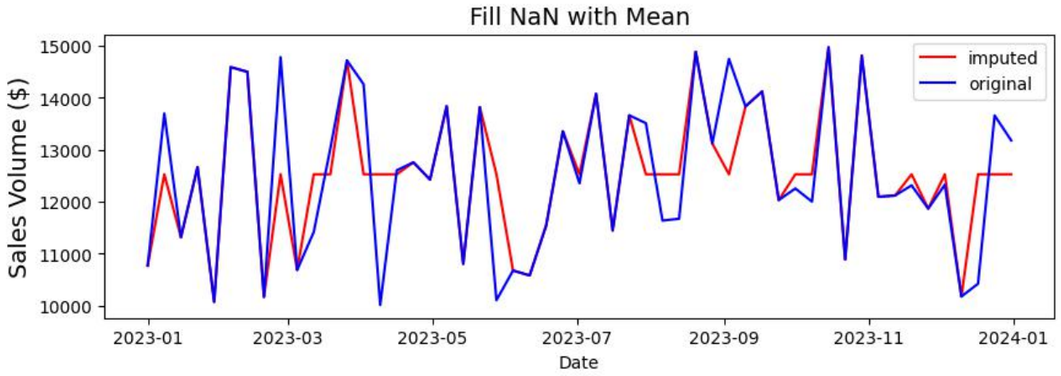
Time Series Imputer

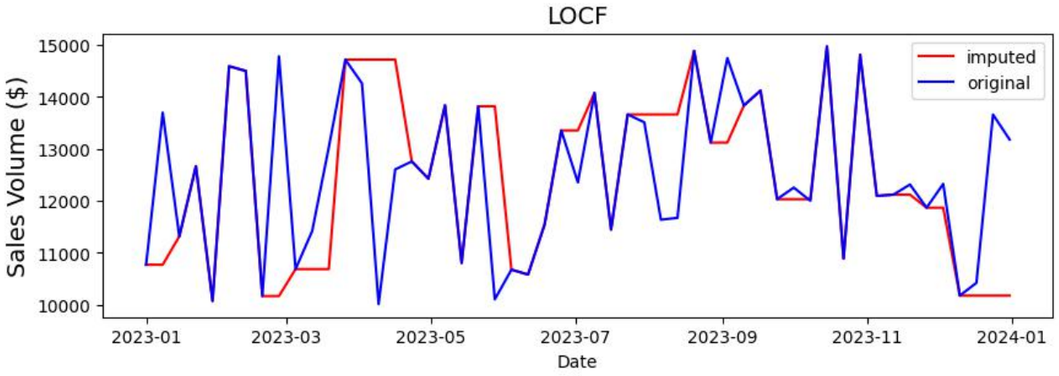
```
# LOCF Imputer
df["Sales"].ffill()
# NOCB Imputer
df["Sales"].bfill()
```

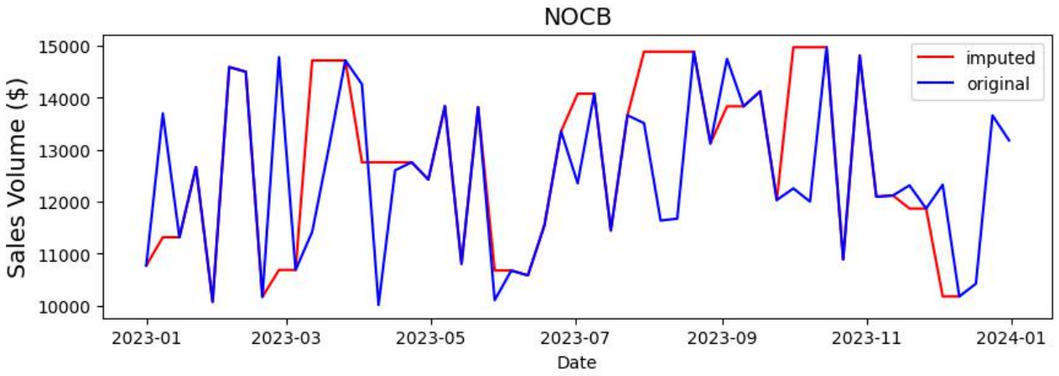
- Also called forward and backward fill.

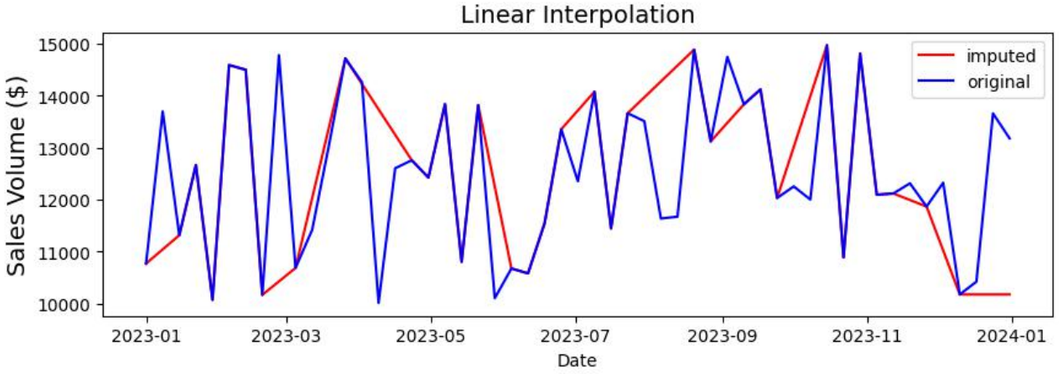
Imputation - Univariate - LOCF & NOCB (for Time Series Data)











Lab - Imputation for Time Series Data

```python
import pandas as pd
import numpy as np
# Create a date range
date_range = pd.date_range(start='2023-01-01', end='2023-12-31', freq='W')
# Generate random sales data
np.random.seed(42)  # For the same data
sales_data = np.random.uniform(12000, 15000, size=len(date_range)).round(2)

# Setting missing data
num_rows = len(sales_data)
num_sales_missing = 10
missing_sales_indices = np.random.choice(num_rows, size=min(num_sales_missing, num_rows), replace=False)
sales_data[missing_sales_indices] = np.nan

# Create the DataFrame
time_series_df = pd.DataFrame({
    'Date': date_range,
    'Sales': sales_data
})

# Set the Date column as the index
time_series_df.set_index('Date', inplace=True)
time_series_df.head(10)
```

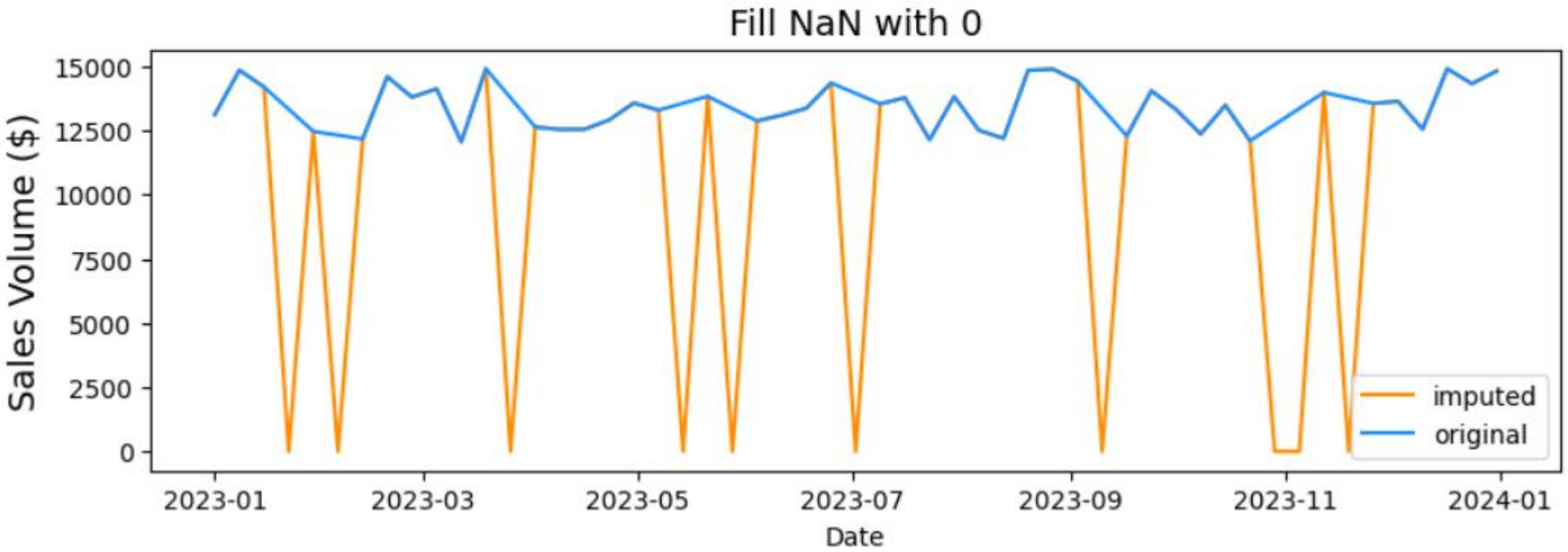
	Sales
Date	
2023-01-01	13123.62
2023-01-08	14852.14
2023-01-15	14195.98
2023-01-22	NaN
2023-01-29	12468.06
2023-02-05	NaN
2023-02-12	12174.25
2023-02-19	14598.53
2023-02-26	13803.35
2023-03-05	14124.22

Lab - Imputation for Time Series Data

```
# Impute missing data with 0
imputed = time_series_df.copy()
imputed['Sales'] = imputed['Sales'].fillna(0)
```

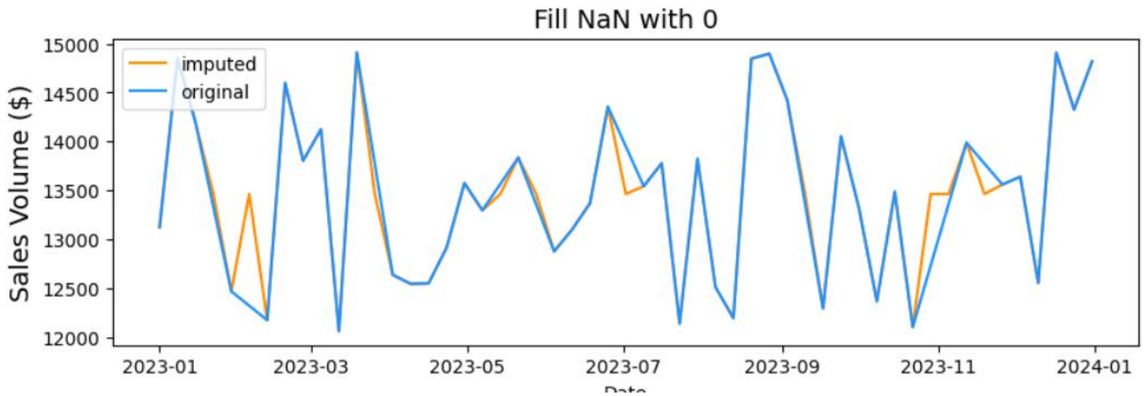
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 3))
sns.lineplot(x=df.index, y=imputed['Sales'], color='darkorange', label='imputed')
sns.lineplot(x=df.index, y=time_series_df['Sales'], color='dodgerblue', label='original')
plt.title('Fill NaN with 0', fontsize=14)
plt.ylabel('Sales Volume ($)', fontsize=14)
plt.legend()
plt.show()
```

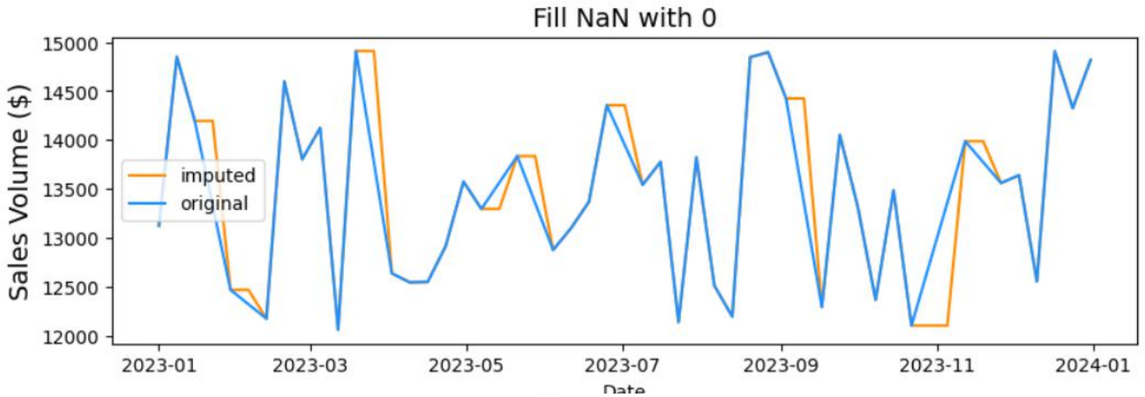


Lab - Imputation for Time Series Data

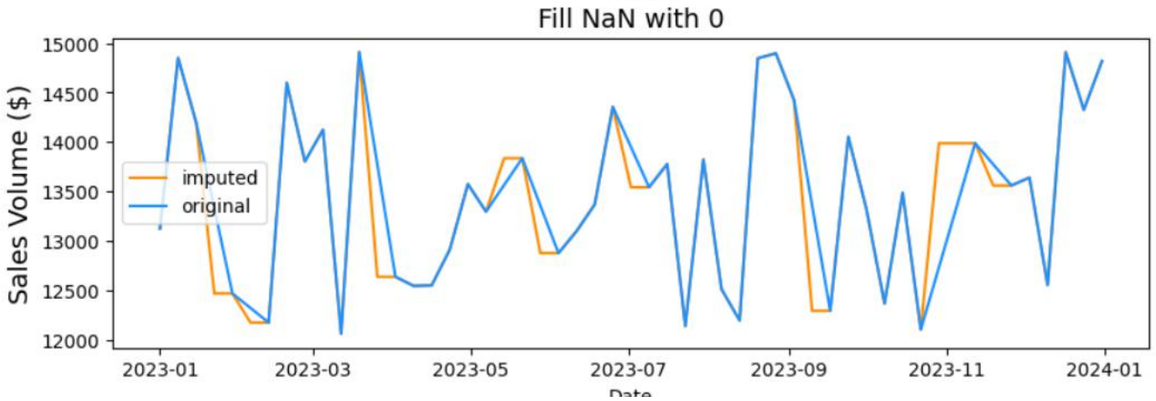
```
# Impute missing data average
imputed = time_series_df.copy()
avg_sales = time_series_df['Sales'].mean()
imputed['Sales'] = imputed['Sales'].fillna(avg_sales)
```




```
# Impute missing data using ffill
imputed = time_series_df.copy()
imputed['Sales'] = imputed['Sales'].ffill()
```




```
# Impute missing data using bfill
imputed = time_series_df.copy()
imputed['Sales'] = imputed['Sales'].bfill()
```



Imputation - Multivariate - Statistical - Bayesian Approach

Multivariate Imputation by Chained Equations (MICE)

- It works by iteratively imputing missing values using a series of conditional models, treating each variable with missing data as a dependent variable in turn.

Example:

- In a medical study, patient cholesterol levels are missing but can be predicted based on age, weight, and diet.

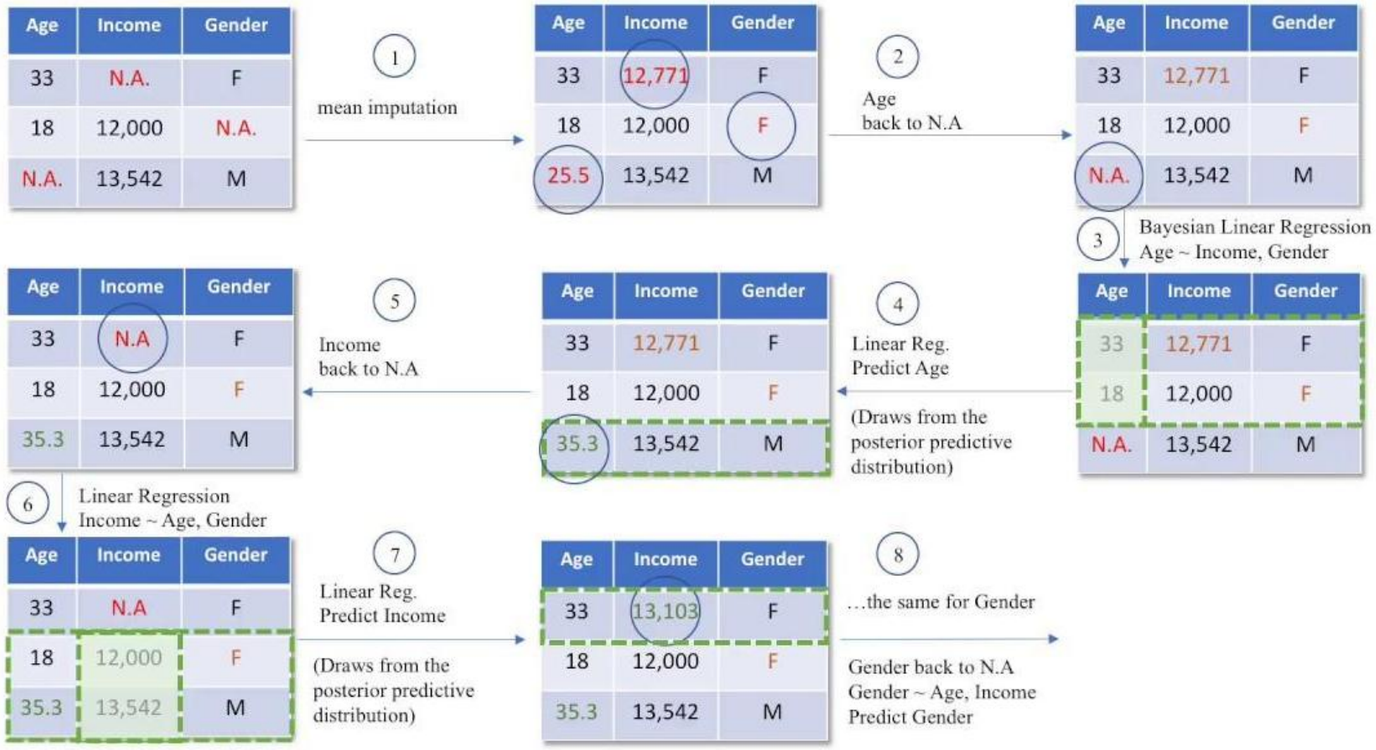
Limitations:

- Computationally expensive, especially for large datasets with many missing values.
- May not work well for highly sparse data, where too many values are missing.

MICE Imputer

```python
from sklearn.impute import IterativeImputer

# Use IterativeImputer (MICE)
imputer = IterativeImputer(max_iter=10, random_state=0)
df['Sales'] = imputer.fit_transform(df[['Sales']])
```

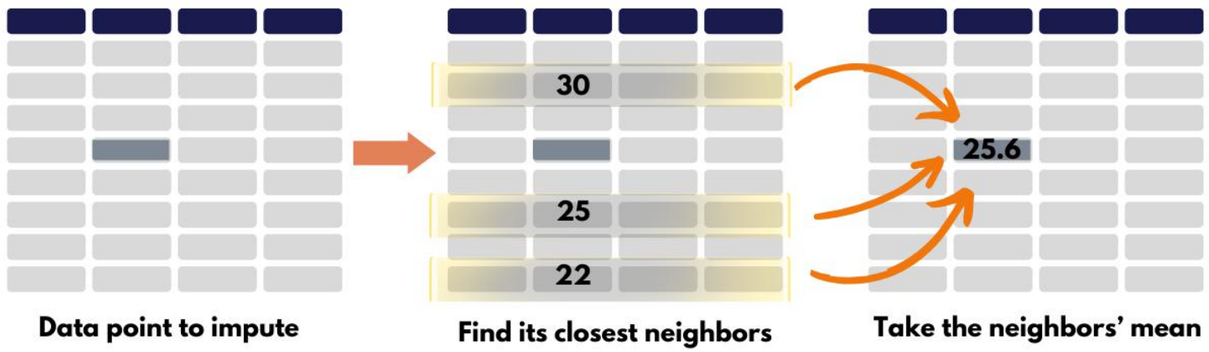


Imputation - Multivariate - Machine Learning - KNN

K-NN imputes the missing value using the value from the closest *k*-neighbors.

The **flexibility** of this method allows for using **diffrent distance functions** and the **various number of neighbors**.

In cases where the missing data follows the MAR mechanism and there is no prior knowledge about its distribution, but information can be gathered from the observed data, K-NN imputation becomes a suitable choice.

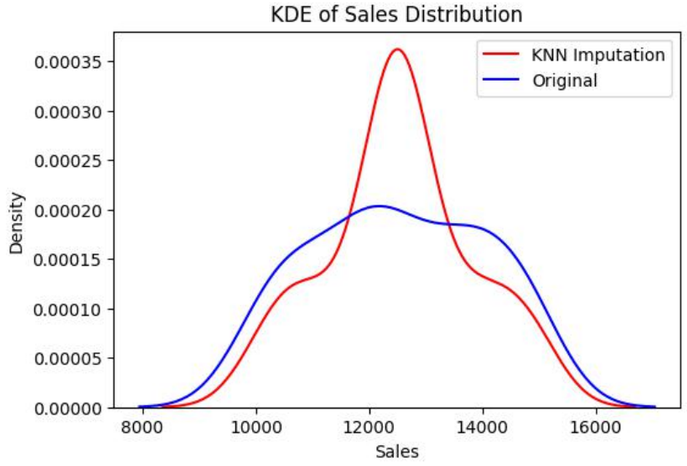


KNN Imputer

```python
from sklearn.impute import KNNImputer

# Create a KNNImputer object
imputer = KNNImputer(n_neighbors=3)

# Fit and transform the imputer on the 'Sales' column
df['Sales'] = imputer.fit_transform(df)
```



Imputation - Multivariate - Machine Learning - Tree

In general, Tree-based models are well-suited for handling missing data because they can naturally split data based using Entropy, Information Gain, or Gini Index on the available features and infer missing values using learned patterns.

Tree-based methods show good performance in handling MCAR and MAR values, and they can also handle MNAR values to some extent.

MissForest (Iterative Random Forest Imputation)

- An advanced iterative imputation method using Random Forests.
- Replaces missing values iteratively, refining estimates at each step.
- Works well for both numerical and categorical data.

MissForest Imputer

```
# pip install MissForest

from missforest import MissForest
# Initialize the imputer
imputer = MissForest()

# Specify categorical columns
categorical_columns = train.select_dtypes('O').columns

# Fit and transform the data
df_imputed = imputer.fit_transform(train, categorical=categorical_columns)
```

Methods That Do NOT Handle Missing Data Directly

Linear Regression / Logistic Regression

- These models do not accept missing values and require imputation before training.

Support Vector Machines (SVM)

- Cannot handle missing values directly; imputation is necessary.

Neural Networks (MLP, CNN, RNN, Transformer)

- Most deep learning frameworks require complete data.
- Some techniques (e.g., Variational Autoencoders for missing data) can be adapted.

k-Means Clustering

- Distance calculations require complete data.

Linear Discriminant Analysis (LDA) / Quadratic Discriminant Analysis (QDA)

- Require complete data.

Most Scikit-Learn Implementations

- Standard implementations of models in Scikit-Learn (except trees) require missing values to be imputed.

Methods That Can Handle Missing Data Natively

Decision Trees (CART, Random Forest, Gradient Boosting)

- Many tree-based models (e.g., Random Forest, XGBoost, LightGBM, CatBoost) can handle missing values by:
- Assigning missing values to separate branches.
- Using surrogate splits (alternative splits when values are missing).

k-Nearest Neighbors (KNN)

- Can work with missing data if a proper distance metric is used (e.g., only computing distances using available features).
- Scikit-learn's KNeighborsClassifier and KNeighborsRegressor require imputation beforehand.

Naïve Bayes

- Some implementations can handle missing values by ignoring missing attributes during probability estimation.

Principal Component Analysis (PCA) Variants

- Some PCA techniques (e.g., Probabilistic PCA, Iterative PCA) can handle missing values.

Expectation-Maximization (EM) Algorithm

- Iteratively estimates missing data values using distributions.

Summary of Imputations methods, according to the missing mechanisme

Missing Type	Best Handling Methods	Avoid
MCAR (Missing Completely at Random)	• Listwise deletion, Mean/Median/Mode, KNN Imputation, MICE	• If too much data is lost in deletion
MAR (Missing at Random)	• Multiple Imputation (MICE), Regression, KNN, Random Forest	• Mean/Median imputation (if relationships exist)
MNAR (Missing Not at Random)	• Domain knowledge, External data, Indicator Variables, Model-Based Approaches (Heckman Model)	• Standard imputation (may introduce bias)

Summary of Statistical based and Machine Learning-based Imputation Method

Imputation Method	Numerical	Categorical	MCAR	MAR	MNAR
Listwise Deletion	✓	✓	✓	×	×
Pairwise Deletion	✓	✓	✓	×	×
Mean/Median	✓	×	✓	×	×
Mode	×	✓	✓	×	×
LOCF & NOCB	✓	✓	✓	×	×
Maximum Likelihood	✓	×	✓	✓	×
Matrix Completion	✓	✓	✓	✓	✓
Bayesian Approach	✓	×	✓	✓	✓
Regression	✓	×	✓	✓	×
K-Nearest Neighbour	✓	✓	✓	✓	×
Tree Based	✓	✓	✓	✓	✓
SVM Based	✓	×	✓	✓	✓
Clustering Based	✓	✓	✓	×	×

Zhou, Youran, Sunil Aryal, and M. Reda Bouadjenek. "A Comprehensive Review of Handling Missing Data: Exploring Special Missing Mechanisms." arXiv preprint arXiv 2404 (2024).

Missing Data vs. Non Data

- Non-Data or Missing by Design
  - Non data occurs intentionally because certain values do not apply to a specific subgroup within the population.
- Why It Happens?:
  - Some variables are only relevant under specific conditions, so they are left blank by design.
- Example:
  - In a medical study, if a questionnaire asks "Are you pregnant?", the follow-up question about trimester is missing for male participants—because it does not apply.
  - In a survey on income, questions about retirement benefits may be missing for respondents under 18 years old.
- Drop non-data cases if they distort analysis: Example, removing incorrect records.

Sex	Age	Pregnant
Female	25	Yes
Female	30	No
Male	40	Missing (N/A)
Female	35	No
Male	28	Missing (N/A)
Female	22	Yes
Male	50	Missing (N/A)

References

https://suparnachowdhury.medium.com/the-art-of-feature-engineering-handling-missing-values-fd6b4290a99e
https://fri-datascience.github.io/course_ids/handbook/missing-data.html#practice-problems-10
A Comprehensive Review of Handling Missing Data: Exploring Special Missing Mechanisms
Python Feature Engineering Cookbook
https://medium.com/@shiptechamit/missing-data-part-2-understanding-missing-values-in-pyhton-c7faae023980
https://www.kaggle.com/code/parulpandey/a-guide-to-handling-missing-values-in-python#Finding-reason-for-missing-data-using-a-Heatmap
https://medium.com/@SelinKarabulut/a-comprehensive-analysis-of-mechanisms-causing-missing-data-and-how-to-handle-it-4c26bec230ae